

RISC-V Pipeline

RISC-V Overview

RISC-V is an **Instruction Set Architecture (ISA)**. An ISA defines:

- The set of machine instructions
- Instruction formats and encodings
- Programmer-visible registers
- Memory addressing rules

RISC-V follows **RISC design principles**:

- Fixed-length instructions (32-bit in RV32)
- Simple instruction formats
- Load/store architecture
- Large register file

The main goal of pipelining is to increase the **throughput** of the processor by allowing multiple instructions to be in different stages of execution simultaneously, much like an assembly line. An ideal 5-stage pipeline aims to complete **one instruction per clock cycle** (**CPI = 1**) after the pipeline is initially filled.

Instruction Pipelining

Instruction pipelining is a microarchitectural technique that allows:

- Overlapping execution of multiple instructions
- One instruction to enter the pipeline every cycle (ideal case)

Each instruction is divided into **sequential stages**. Pipeline registers separate stages and store intermediate results.

5-Stage RISC-V Pipeline

Stage	Abbreviation	Function
1	IF	Instruction Fetch
2	ID	Instruction Decode & Register Fetch
3	EX	Execute / Address Calculation

4	MEM	Memory Access
5	WB	Write Back

Pipeline registers: To ensure an instruction's data moves correctly from one stage to the next, we use **Pipeline Registers** (buffers) placed between the stages. These registers hold the intermediate results and control signals needed by the next stage.

- **IF/ID:** Stores the fetched **Instruction** and the incremented **PC+4**.
- **ID/EX:** Stores the decoded control signals, register values (A and B), immediate value, destination register ID (r_d), and PC+4.
- **EX/MEM:** Stores the ALU result/address, the value to be stored (for a store instruction), control signals, and the destination register ID.
- **MEM/WB:** Stores the final result (either from ALU or memory), control signals, and the destination register ID

Stage	Abbreviation	Primary Actions	Datapath Components Involved
Instruction Fetch	IF	1. Fetch the instruction from Instruction Memory at the address stored in the Program Counter (PC) .	PC, Instruction Memory, Adder (for PC + 4)
Instruction Decode	ID	1. Decode the instruction (determine its type, operation, and operands). 2. Read up to two register operands from the Register File . 3. Calculate the immediate value (if needed). 4. Calculate the potential branch target address .	Register File, Decoder/Control Unit, Immediate Generator

Execute	EX	1. Perform the main ALU operation (e.g., addition, subtraction, logic). 2. For load/store instructions, calculate the effective memory address . 3. For branch instructions, evaluate the branch condition (e.g., if registers are equal).	ALU, Multiplexers (to select ALU inputs), Branch Comparator
Memory Access	MEM	1. Access Data Memory: Read data from memory (Load instruction) or Write data to memory (Store instruction). 2. Non-memory instructions (like R-type) simply pass through this stage.	Data Memory
Write Back	WB	1. Write the result back to the Register File . The result could come from the ALU (EX stage) or from Data Memory (MEM stage).	Register File (write port), Multiplexer (to select data source)

Register File vs Pipeline Registers

Feature	Register File (RF)	Pipeline Registers (Buffers)
Primary Purpose	Programmer-Visible Storage. Holds data and addresses that the program directly manipulates.	Structural Storage. Temporarily holds the intermediate state of an instruction as it moves between pipeline stages.

Visibility	Visible to the programmer (via Instruction Set Architecture). The instruction explicitly names these registers.	Invisible to the programmer. These are purely internal hardware components necessary for the pipeline's structure.
Location in Datapath	Found in the Instruction Decode (ID) stage (for reading operands) and the Write Back (WB) stage (for writing results).	Located between every pipeline stage (e.g., IF/ID, ID/EX, EX/MEM, MEM/WB).
Key Role	State of the program. Stores the variables, function arguments, return values, and memory addresses used by the running program.	Isolation and Synchronization. Isolate the stages, ensuring the output of one stage is held constant and passed correctly to the next stage on the clock edge, preventing data corruption.
Inputs/Outputs	Typically has two read ports (for source operands, r_s1, r_s2) and one write port (for destination register, r_d).	Input is the entire output bundle of the preceding stage; output is the input bundle for the following stage. They are simple, wide registers.
Control	Program-driven. Controlled by the register addresses (r_s1, r_s2, r_d) specified <i>within the instruction</i> being decoded.	Clock-driven (synchronous). Automatically updated on the clock edge, but can be controlled by a stall/flush signal (e.g., Load-Use Hazard or Branch Hazard) to disable the clock enable and insert a NOP.

Pipeline Hazards

Pipelining creates three main types of *hazards*—situations where the next instruction cannot execute in the following clock cycle, potentially leading to incorrect results.

A. Data Hazards (RAW: Read After Write)

This is the most common hazard. It occurs when an instruction needs to read a register's value, but a previous instruction in the pipeline hasn't finished writing the new value back to the register file yet.

- **Example:**
 - ADD x1, x2, x3 (Writes to x1 in WB stage, Cycle 5)
 - SUB x4, x1, x5 (Reads x1 in ID stage, Cycle 2)
 - SUB will read the *old* value of x1 because the ADD instruction's result is not written back until Cycle 5.

Solution: Data Forwarding (or Bypassing)

- **Concept:** Instead of waiting for the result to be written back to the Register File, the hardware detects the dependency and directly **forwards (bypasses)** the result from the intermediate pipeline registers (**EX/MEM** or **MEM/WB**) back to the **ALU input multiplexers** in the **EX** stage.
- **Implementation:** Requires a dedicated **Forwarding Unit** that checks the destination register (r_d) of the instructions in the EX and MEM stages against the source registers (r_{s1}, r_{s2}) of the instruction in the ID stage.

Data Hazard requiring a Stall (Load-Use Hazard)

- A load instruction (LW) only retrieves its data from memory in the **MEM** stage. If the very next instruction tries to use that loaded register in the **EX** stage, even forwarding is too late.
- **Solution: Pipeline Stall (or Bubble)**
 - The **Hazard Detection Unit** detects the load-use dependency.
 - It inserts a **NOP** (No Operation) instruction into the **ID/EX** register and freezes the **IF** and **ID** stages for one clock cycle. This effectively creates a "bubble," delaying the dependent instruction by one cycle, which is enough time for the load data to become available for forwarding from the MEM/WB register.

B. Control Hazards (Branch Hazards)

These occur when the processor fetches an instruction that should not be executed because of a conditional branch (BEQ, BNE, etc.) or a jump instruction (JAL).

- The next instruction is fetched in **IF** *before* the branch decision is made in the **EX** stage.

Solution 1: Predict Not Taken (Simplest)

- **Concept:** Assume the branch will **not** be taken and continue fetching instructions sequentially (PC+4).
- **If correct:** No penalty.
- **If incorrect (Branch is taken):** The incorrectly fetched instructions (usually the two after the branch) must be **flushed** (overwritten with NOPs) from the pipeline, and the PC must be set to the correct branch target address. This incurs a **branch penalty** (typically 2 cycles for a 5-stage pipeline).

C. Structural Hazards

This occurs when two different instructions try to use the **same hardware resource** at the same time.

- **Standard RISC-V 5-stage design typically avoids this by:**
 1. Using separate **Instruction Memory (IMEM)** and **Data Memory (DMEM)** (or separate caches). This is a **Harvard Architecture** concept. Without this, an instruction fetch (IF) would conflict with a load/store (MEM).
 2. The **Register File** has separate read ports (used in ID) and a dedicated write port (used in WB) to handle simultaneous reads and writes.
-

Project Implementation Steps (for a Beginner)

Step 1: Design the Single-Cycle Processor (The Foundation)

- Start by implementing a non-pipelined, single-cycle RISC-V processor. This will give you the correct datapath components (ALU, Register File, Control Unit) and their control signals.

Step 2: Introduce Pipelining

- Break the single-cycle design into the five stages.
- Insert the four **Pipeline Registers (IF/ID, ID/EX, EX/MEM, MEM/WB)** between the stages.
- **Crucial:** All control signals must also travel through the pipeline registers so that the correct control signals arrive at the datapath component at the right clock cycle.

Step 3: Implement Hazard Control

- **Hazard Detection Unit:** Place this logic in the **ID** stage. It checks for **Load-Use** hazards and signals a stall.
- **Forwarding Unit:** Place this logic in the **EX** stage. It checks for other **Data Hazards** and controls the ALU input multiplexers.
- **Control/Branch Logic:** Handle branch instructions by flushing the pipeline (turning the IF/ID and ID/EX registers into NOPs) when a branch is taken, and updating the PC.

Step 4: Verification

- Write simple RISC-V assembly programs (e.g., Fibonacci, array sum) that specifically test your hazard logic (e.g., an ADD instruction immediately followed by an instruction using its result).
- Use a simulation tool (like GTKWave for Verilog/VHDL) to observe the signals in the pipeline registers cycle-by-cycle to ensure your instructions execute correctly and hazards are resolved.

Why is a program counter updated by +4?

The Program Counter (PC) updates by +4 because of two key architectural facts about the base RISC-V instruction set:

1. Fixed Instruction Length

- **RISC Principle:** A key principle of RISC architectures like RISC-V (specifically the RV32I or RV64I base instruction sets) is that most instructions have a fixed length.
- **The Length:** In the base RISC-V specification, **every instruction is 32 bits (4 bytes) long.**
- **The Goal:** The processor fetches one entire instruction at a time from memory.

2. Byte Addressing

- **Memory Standard:** RISC-V (like almost all modern CPUs) uses **byte addressing**. This means that every single byte in memory has a unique address.
- **Sequential Access:**
 - If the current instruction starts at memory address N, the processor reads the 4 bytes from N to N+3 to get the complete 32-bit instruction.
 - Since the next instruction immediately follows the current one in memory (for sequential execution), it must start at the address right after the current instruction ends, which is N+4.

PC Update in the IF Stage

In your 5-stage pipeline project, the PC update happens in the **Instruction Fetch (IF) stage**

When an instruction is fetched at the beginning of the clock cycle, the processor calculates the address of the *next* instruction in parallel. This is done by a simple adder:

$\text{New PC} = \text{Current PC} + 4$

This calculated PC +4 value is then stored in the **IF/ID pipeline register** (along with the fetched instruction) to be used in the next clock cycle unless a **branch or jump** instruction is detected.

What about Branches and Jumps?

The +4 update is only the **default, sequential PC update**. It is the path taken when the program flows straight through (i.e., no branch is taken).

If the processor encounters a branch (BEQ, JAL, etc.) in the **EX** stage:

- The PC is updated by a different value (the **branch target address**), and the speculative +4 instruction that was already fetched (and is now in the ID or EX stage) is **flushed** (turned into a NOP) to prevent incorrect execution.

Code explanation

1. pipeline.h

Includes and Global Definitions

```
#include <array>
#include <cstdint>
#include <string>
#include <vector>
#include <iostream>
```

```
using std::array;
using std::string;
// ... (using directives)
```

- **Purpose:** Standard C++ library inclusions to support the data structures used.
 - `<cstdint>`: Essential for portable integer types like `uint32_t` (32-bit unsigned) and `int32_t` (32-bit signed), which directly map to RISC-V register sizes and memory addresses.
 - `<array>/<vector>`: Used for the Register File and Data Memory
 - `<string>`: Used to store the instruction's mnemonic (`instr_str`) for easy visualization.

Pipeline Register Structs (IF_ID to MEM_WB)

They model the physical register placed between the combinatorial logic of each stage.

A. struct IF_ID (Instruction Fetch / Instruction Decode)

```
struct IF_ID {
    uint32_t pc = 0;
    uint32_t instr = 0;
    string instr_str = "NOP";
    bool valid = false;
};
```

- **Purpose:** Holds the raw information fetched from Instruction Memory.
- **pc:** The address of the instruction. Crucial for calculating branch targets later.
- **instr:** The raw 32-bit binary instruction.
- **instr_str:** The instruction's mnemonic (e.g., "ADD x1, x2, x3"), added for display purposes.
- **valid:** A flag to indicate if this register holds a real instruction or a NOP/flushed content.

B. struct ID_EX (Instruction Decode / Execute)

```
struct ID_EX {
    // ... basic tracking: pc, instr, instr_str
```



```

int32_t rs1_val = 0; // Value of the first source register (read from RF)
int32_t rs2_val = 0; // Value of the second source register (read from RF)
int32_t imm = 0;    // Sign-extended immediate value

uint8_t rd = 0;     // Destination register ID (e.g., 5 for x5)
uint8_t rs1 = 0;    // Source 1 register ID
uint8_t rs2 = 0;    // Source 2 register ID

// Control Signals for downstream stages (EX, MEM, WB)
bool useImm = false; // Control for ALU input mux (rs2_val vs imm)
bool memRead = false;
bool memWrite = false;
bool regWrite = false;
bool memToReg = false;

// ALU Control Signal
enum ALUOp_t { /* ... */ } aluOp = ALU_NONE;

bool valid = false;
};

```

- **Purpose:** Contains all necessary data and **control signals** for the instruction to complete the remaining three stages.
- **Register Values (rs1_val, rs2_val):** Read from the Register File in the ID stage and carried forward to be used as ALU inputs in EX.
- **Register IDs (rd, rs1, rs2):** Carried forward for forwarding logic and the final WB write.
- **Control Signals:** These signals are generated in the ID stage and are crucial. They travel with the instruction through all remaining buffers to control the EX, MEM, and WB stages when the instruction arrives there.
- **ALUOp_t:** An enumeration defining the operation the ALU should perform in the EX stage.

C. struct EX_MEM (Execute / Memory Access)

```

struct EX_MEM {
    // ... basic tracking: pc, instr, instr_str
    int32_t alu_result = 0; // The result of the ALU (e.g., for R-type or address for
    Load/Store)
    int32_t rs2_val = 0;    // Value of rs2, needed for a Store instruction (to be written to
    memory)
    uint8_t rd = 0;        // Destination register ID

    // Control Signals passed to MEM and WB stages
    bool memRead = false;
    bool memWrite = false;
    bool regWrite = false;
    bool memToReg = false;
};

```

```

    bool valid = false;
};

```

- **Purpose:** Holds the result of the EX stage and the data needed for memory operations.
- **alu_result:**
 - For R-Type instructions (ADD, SUB, etc.), this is the final result.
 - For Load/Store instructions, this is the calculated effective **memory address**.
- **rs2_val:** Passed down **only** for Store instructions, as this is the data that needs to be written to memory in the MEM stage.

D. struct MEM_WB (Memory Access / Write Back)

```

struct MEM_WB {
    // ... basic tracking: pc, instr, instr_str
    int32_t mem_data = 0; // Data read from Data Memory (only for Load instructions)
    int32_t alu_result = 0; // Final result from ALU (for R-type or Branch/Jump)
    uint8_t rd = 0;       // Destination register ID

    // Control Signals for the WB stage
    bool regWrite = false;
    bool memToReg = false;

    bool valid = false;
};

```

- **Purpose:** Holds the final result ready to be written back to the Register File.
- **mem_data:** The result of a Load operation performed in the MEM stage.
- **alu_result:** The result of non-memory operations (already finalized).
- **memToReg:** This control signal determines which value (either mem_data or alu_result) is written back to the register.

PipelineState and RegisterFile

A. struct PipelineState (Snapshot)

```

struct PipelineState {
    // ... cycle, pc
    string IF_instr = "NOP";
    // ... ID_instr, EX_instr, MEM_instr, WB_instr
    array<int32_t, 32> regs{};
};

```

- **Purpose:** A utility structure primarily for the **Qt GUI**. It captures the entire state of the simulator at a single clock edge, making it easy to display in a single frame.
- **IF_instr through WB_instr:** Used to show which instruction mnemonic is currently occupying each of the five stages.
- **regs:** A copy of the Register File state, used to update the register display table.

B. class RegisterFile

```
class RegisterFile {
public:
    RegisterFile();
    int32_t read(uint8_t reg) const;
    void write(uint8_t reg, int32_t value);
    // ... snapshot(), dump()

private:
    array<int32_t, 32> regs{};
};
```

- **Purpose:** Models the 32 General Purpose Registers of the RISC-V architecture.
- **read(uint8_t reg):** Called during the **ID stage** to get the source operand values (rs1_val, rs2_val). It must enforce that reg 0 always reads as 0 (the RISC-V zero register).
- **write(uint8_t reg, int32_t value):** Called during the **WB stage** to finalize the instruction's result. Must enforce that reg0 is never written.

Memory Class

```
class Memory {
public:
    Memory(size_t bytes = 1024);
    void sw(uint32_t address, int32_t value);
    int32_t lw(uint32_t address) const;
    // ... dumpWords()

private:
    vector<uint8_t> data;
};
```

- **Purpose:** Models the byte-addressable Data Memory.
- **data:** The underlying storage. Using vector<uint8_t> allows for byte-level access, aligning with the byte-addressing scheme.
- **lw(address):** Implements the **Load Word** operation (reads 4 bytes from memory). Called during the **MEM stage** when memRead.
- **sw(address, value):** Implements the **Store Word** operation (writes 4 bytes to memory). Called during the **MEM stage** when memWrite is true.

PipelineSimulator Class

```
class PipelineSimulator {
public:
    // ... constructor, loadProgram()

    // The five stages (called by step())
```

```

void fetch_instruction(const string &instr, uint32_t pc_val = 0);
void decode();
void execute();
void mem_access();
void writeback();

void step(); // Single pipelined cycle
// ... snapshot()

// Public for UI visibility
vector<string> instr_mem; // Instruction Memory (Program Code)
uint32_t pc = 0;         // Program Counter

RegisterFile regs;
Memory mem;

private:
    // Internal pipeline registers (instances of the structs above)
    IF_ID if_id;
    ID_EX id_ex;
    EX_MEM ex_mem;
    MEM_WB mem_wb;

    // Parsers (for loading assembly into the simulator)
    static bool parseLoadStore(const string &s, /* ... */);
    static bool parseStore(const string &s, /* ... */);
};

```

2. **Purpose:** The central controller that manages the clock, orchestrates the five stages, and holds the state of the entire system.
3. **instr_mem:** Models the Instruction Memory, holding the program instructions (as strings, which will be converted to 32-bit words).
4. **step():** The most crucial function. It executes the stages in a specific order (typically WB to MEM to EX to ID to IF) and advances the data between the internal pipeline register instances (if_id, id_ex, etc.).
5. **Stage Methods (decode(), execute(), etc.):** These functions contain the combinatorial logic for each stage, reading from the previous pipeline register instance and writing the result to the next.
6. **private Pipeline Registers:** Keeping these instances private ensures they can only be manipulated by the controlled stage functions, mimicking hardware integrity.
7. **Parsers:** Utility functions needed to convert human-readable assembly instructions (strings) into the binary format (or equivalent) needed for the simulation.

8. pipeline.cpp
9. mainwindow.h
10. mainwindow.cpp
11. main.cpp